

Capitulo 02

Micrograd

Construindo um Modelo de Linguagem do Zero

Curso bilingue inspirado no LLM101n

Gerado em 29/07/2026

Capítulo 02 — Micrograd (autograd e backpropagation)

Objetivo de aprendizagem: abrir a caixa-preta do `loss.backward()`. Vamos construir, do zero, um motor de **autograd** (diferenciação automática) e usá-lo para treinar uma rede neural de verdade — entendendo exatamente como o gradiente de cada peso é calculado pela **backpropagation**.

Pré-requisitos deste capítulo: o Capítulo 1 (você já viu `loss`, gradiente e gradient descent "de fora"). De matemática, só a ideia de **derivada** como "taxa de variação" — que revisamos já na Seção 1. Programação: vamos usar **classes** em Python; se você nunca criou uma, a Seção 4 explica o suficiente.

Arquivos: - `micrograd.py` — o motor de autograd (a classe `Value`) - `nn.py` — uma mini biblioteca de redes neurais sobre o `Value` - `exercicios.md` — exercícios

1. O problema: como a rede sabe para que lado ir?

No Capítulo 1, treinar a rede foi isto:

```
loss.backward() # <- mágica: de onde saem os gradientes?
W.data += -50 * W.grad
```

A segunda linha é simples: "ande na direção que diminui a loss". Mas ela depende de `W.grad` — o **gradiente** —, e esse número apareceu de graça quando chamamos `loss.backward()`. Este capítulo é sobre **fabricar esse número nós mesmos**.

Derivada em 30 segundos

A **derivada** de uma quantidade em relação a outra responde: "se eu mexer um tiquinho nesta entrada, o quanto a saída muda, e para que lado?"

Formalmente, é o limite

$$df/dx = \lim (h \rightarrow 0) [f(x + h) - f(x)] / h$$

mas a intuição é o que importa: a derivada é a **inclinação** (positiva = "subindo", negativa = "descendo") e a **sensibilidade** (quão forte). Se $df/dx = 3$, aumentar x em `0.001` aumenta f em ~ 0.003 . Se $df/dx = -2$, aumentar x diminui f .

Gradiente = todas as derivadas de uma vez

Numa rede com milhares de pesos, o **gradiente** é só o conjunto de todas as derivadas da loss em relação a cada peso: $\partial loss / \partial w$ para cada w . Cada uma diz como mexer **aquele** peso para baixar a loss. É exatamente o `W.grad` do Capítulo 1.

O desafio: calcular isso à mão para milhares de pesos é inviável. A solução é a **backpropagation** — um jeito mecânico e barato de obter todas as derivadas de uma vez. E ela se apoia em uma única regra do cálculo: a **regra da cadeia**.

2. A regra da cadeia (chain rule), a chave de tudo

Imagine três engrenagens encaixadas. A primeira gira a segunda na proporção de 2:1; a segunda gira a terceira na proporção de 3:1. Quão rápido a terceira gira em relação à primeira? $2 \times 3 = 6$. Você **multiplica as taxas locais ao longo do caminho**.

Isso é a regra da cadeia. Se y depende de u , que depende de x :

$$dy/dx = (dy/du) \cdot (du/dx)$$

A backpropagation é só isto aplicado repetidamente: para saber o efeito de um peso lá no começo sobre a loss lá no fim, **multiplicamos as derivadas locais de cada passo no caminho de volta**. Cada operação só precisa saber a sua própria derivada local ("eu giro o próximo na proporção X"). Juntando o caminho todo, sai o gradiente.

A grande sacada: se cada operaçãozinha (+, *, tanh, ...) souber a sua derivada local, conseguimos compor essas peças em qualquer expressão gigante e ainda assim calcular todos os gradientes — só propagando de trás para frente.

3. O grafo de computação

Toda expressão pode ser desenhada como um **grafo**: as folhas são as entradas, e cada operação é um nó que combina resultados anteriores. Veja $d = a * b + c$:

```
a --+
      (*) --> e --+
b --+ (+) --> d
c -----+
```

A computação normal anda da esquerda para a direita (**forward pass**): calcula e , depois d . A backpropagation anda da direita para a esquerda (**backward pass**): parte de d , distribui o gradiente para e e c , depois de e para a e b .

Para fazer isso no código, cada valor precisa **lembrar de onde veio** — quais nós o geraram e por qual operação. É essa "memória" que vamos construir agora.

4. A classe `Value`

O coração do micrograd é uma classe que embrulha **um único número** e participa do grafo. Se você nunca usou classes: pense numa classe como um molde que junta dados (o número e seu gradiente)

com comportamentos (somar, multiplicar...). Cada `Value` criado é um "objeto" feito nesse molde.

```
class Value:
    def __init__(self, data, _children=(), _op=""):
        self.data = data # o número em si
        self.grad = 0.0 # d(loss)/d(este value); começa zerado
        self._backward = lambda: None # função que propaga o grad aos pais
        self._prev = set(_children) # de quais Values este nasceu
        self._op = _op # rótulo da operação (debug)
```

Dois campos são "públicos" e te interessam: `data` (o valor) e `grad` (a derivada da loss em relação a ele). Os três com underline são a maquinaria interna do autograd: `_prev` guarda os "pais" no grafo, `_op` é só um rótulo, e `_backward` é uma função que sabe distribuir o gradiente deste nó para os pais.

5. Cada operação conhece sua derivada local

Aqui mora a inteligência. Ao sobrecarregar `+` e `*`, cada operação faz **duas** coisas: calcula o resultado (forward) e registra como o gradiente volta (backward).

Soma

```
def __add__(self, other):
    other = other if isinstance(other, Value) else Value(other)
    out = Value(self.data + other.data, (self, other), "+")

    def _backward():
        self.grad += 1.0 * out.grad
        other.grad += 1.0 * out.grad
    out._backward = _backward
    return out
```

A derivada local da soma é **1 para cada parcela** ($d(a+b)/da = 1$). Então a soma apenas **repassa** o gradiente que chega (`out.grad`) igualzinho para os dois pais. É o "tubo de distribuição": a soma espalha o gradiente sem alterá-lo.

Multiplicação

```
def __mul__(self, other):
    other = other if isinstance(other, Value) else Value(other)
    out = Value(self.data * other.data, (self, other), "*")

    def _backward():
        self.grad += other.data * out.grad
        other.grad += self.data * out.grad
    out._backward = _backward
    return out
```

Aqui $d(a \cdot b)/da = b$ e $d(a \cdot b)/db = a$ — cada pai recebe o gradiente **multiplicado pelo valor do outro**. Faz sentido: se `b` é grande, mexer em `a` tem efeito grande.

Por que += e não =? (o detalhe que todo mundo erra)

Repare que **acumulamos** com `+=`. Se um mesmo `Value` é usado em dois lugares (ex.: `b = a + a`), o gradiente chega por **dois caminhos** e os efeitos **se somam** (é a regra da soma das derivadas). Usar `=` apagaria a primeira contribuição e daria o gradiente errado. Por isso, antes de cada `backward`, precisamos **zerar os gradientes** — o famoso `zero_grad()` / `W.grad = None` do Capítulo 1 reaparece aqui, agora com o motivo explícito.

Funções não-lineares: `tanh`, `exp`, `relu`

Uma rede que só soma e multiplica é, no fim, uma única conta linear — incapaz de aprender padrões curvos. Precisamos de uma **não-linearidade**. Usamos a `tanh`, que amassa qualquer número para o intervalo `(-1, 1)`:

```
def tanh(self):
    t = math.tanh(self.data)
    out = Value(t, (self,), "tanh")
    def _backward():
        self.grad += (1 - t**2) * out.grad # d(tanh)/dx = 1 - tanh(x)^2
    out._backward = _backward
    return out
```

O padrão é sempre o mesmo: calcula o valor, e guarda em `_backward` a derivada local multiplicada pelo gradiente que vem de fora. `exp` e `relu` (veja o código) seguem a mesma receita. **É só isso que um framework de deep learning faz** — para centenas de operações, na GPU, mas com exatamente esta estrutura.

6. A `backward()`: percorrer o grafo de trás para frente

Com cada nó sabendo sua derivada local, falta orquestrar: chamar os `_backward()` na **ordem certa**. A regra: só posso processar um nó depois de já ter recebido todo o gradiente que vem "de fora" dele. Isso é uma **ordenação topológica** do grafo.

```
def backward(self):
    topo = []
    visited = set()
    def build_topo(v):
        if v not in visited:
            visited.add(v)
            for child in v._prev:
                build_topo(child)
            topo.append(v)
    build_topo(self)

    self.grad = 1.0 # d(loss)/d(loss) = 1: a semente
    for node in reversed(topo):
        node._backward()
```

Dois detalhes:

1. A semente `self.grad = 1.0`. A derivada da loss em relação a ela mesma é 1. Esse `1.0` é a "gota" de gradiente que entra no topo e vai se distribuindo grafo abaixo pela regra da cadeia.
2. `reversed(topo)`. Construímos a ordem em que cada nó vem *depois* dos seus filhos; invertendo, processamos da loss para as folhas — o sentido do backward.

Veja funcionando

Rode `python micrograd.py`. Ele monta `L = tanh(a*b + c)` e chama `L.backward()`:

```
L = Value(data=0.9993, grad=1.0000)
dL/da = -0.00402...
dL/db = 0.00268...
dL/dc = 0.00134...
```

Os gradientes são pequenos porque a `tanh` está **saturada** (entrada `a*b+c = 4`, bem na parte "achatada" da curva, onde a inclinação é quase zero). Guarde esse efeito — "gradientes que somem em regiões saturadas" é um tema que volta no Capítulo 7.

7. De um número a uma rede neural

Um neurônio biológico recebe sinais, pondera e dispara. O artificial faz o mesmo: soma ponderada das entradas mais um *bias*, passada por uma não-linearidade. Com a classe `Value`, isso são poucas linhas (veja `nn.py`):

```
class Neuron(Module):
    def __init__(self, nin, nonlin=True):
        self.w = [Value(random.uniform(-1, 1)) for _ in range(nin)]
        self.b = Value(0.0)
        self.nonlin = nonlin
    def __call__(self, x):
        act = sum((wi * xi for wi, xi in zip(self.w, x)), self.b) # w·x + b
        return act.tanh() if self.nonlin else act
```

Cada peso `w` e o `b` são `Values` — ou seja, **fazem parte do grafo** e terão gradiente automático. Empilhando neurônios viram uma `Layer`, e empilhando camadas vira um `MLP` (Multi-Layer Perceptron). Esse é, literalmente, o modelo do Capítulo 3.

8. Treinando: o mesmo loop do Capítulo 1, agora "nosso"

A demo em `nn.py` treina um MLP `3 -> 4 -> 4 -> 1` para classificar 4 pontos. O laço de treino é idêntico em espírito ao do Capítulo 1 — forward, backward, update:

```
for step in range(100):
    ypred = [model(x) for x in xs] # forward
    loss = sum((yp - yt)**2 for yp, yt in zip(ypred, ys)) # erro quadrático
    model.zero_grad() # zera (por causa do +=!)
```

```
loss.backward() # NOSSA backpropagation
for p in model.parameters():
    p.data -= 0.05 * p.grad # gradient descent
```

Rode `python nn.py`:

```
parametros no modelo: 41
step 0 | loss 3.092861
step 10 | loss 0.000186
...
loss final = 0.000000
previsoes (alvo entre parenteses):
+1.000 (+1)
-1.000 (-1)
-1.000 (-1)
+1.000 (+1)
```

A loss despenca e as 4 previsões batem com os alvos. **Treinamos uma rede neural com um motor de autograd que escrevemos do zero.** O `loss.backward()` deixou de ser mágica.

9. "Então o PyTorch é isto?" — Sim.

A diferença entre o nosso `Value` e o `Tensor` do PyTorch é **engenharia, não conceito**:

- PyTorch opera sobre **tensores** (arrays n-dimensionais), não um escalar por vez — por isso é milhares de vezes mais rápido (uma matmul de uma vez, na GPU).
- Tem muito mais operações, e roda em GPU.

Mas a alma é idêntica: cada operação registra sua derivada local, e um `backward()` percorre o grafo aplicando a regra da cadeia. Tanto que **os gradientes do nosso micrograd batem com os do PyTorch até a 6ª casa decimal** — você confirma isso no exercício E5. A partir daqui, voltamos a usar PyTorch sabendo exatamente o que ele faz por baixo.

10. Resumo do capítulo

- A **derivada** mede sensibilidade e direção; o **gradiente** reúne as derivadas da loss em relação a todos os pesos.
- A **regra da cadeia** permite obter o gradiente de qualquer expressão multiplicando derivadas locais ao longo do caminho de volta.
- Toda expressão é um **grafo de computação**; cada nó guarda como nasceu (`_prev`) e sua derivada local (`_backward`).
- **Backpropagation** = ordenar o grafo topologicamente e aplicar `_backward()` da loss até as folhas, semeando com `grad = 1.0`.
- Gradientes se **acumulam** (`+=`) -> por isso é preciso **zerar** antes de cada passo.

- Com isso construímos **Neuron -> Layer -> MLP** e treinamos uma rede de verdade. O `loss.backward()` do PyTorch é exatamente este mecanismo, vetorizado.

O que vem no Capítulo 3

Temos um motor de autograd e um MLP. No **Capítulo 03 — N-gram model** voltamos ao problema de gerar nomes do Capítulo 1, mas trocamos o bigrama por um **MLP** que olha *vários* caracteres de contexto — usando PyTorch de verdade, com `matmul`, embeddings e a função de ativação **GELU**. É a ponte do brinquedo para o modelo de verdade.

-> Antes de seguir, faça os [exercícios](#).

Exercícios — Capítulo 02 (Micrograd)

Faça na ordem. Soluções comentadas em [solucoes/](#) — tente antes de olhar.

E1 — Derivada na unha (aquecimento)

Sem código, só caneta: 1. Para $f = a * b$ com $a = 3$, $b = 4$, quanto vale df/da ? E df/db ? 2. Para $d = a*b + c$, quanto vale dd/dc ? Por que a soma "não altera" o gradiente? 3. Explique, com suas palavras, por que `tanh` saturada (entrada grande) produz gradiente quase zero.

E2 — Conferindo com diferença finita

A derivada é $[f(x+h) - f(x)] / h$ para h pequeno. Escolha uma expressão simples com `Value` (ex.: $L = a*b + c$), calcule `a.grad` via `backward()`, e depois compare com a estimativa numérica: recalcule `L` com `a` somado de $h = 1e-6$ e use a fórmula. Os dois valores batem? (Essa é a forma clássica de **testar** um autograd.)

E3 — Adicione uma operação nova

Implemente o método `log()` na classe `Value` (logaritmo natural). 1. Qual a derivada local de `log(x)`? (Dica: $d(\ln x)/dx = 1/x$.) 2. Escreva o `_backward` correspondente, seguindo o padrão de `exp()`. 3. Teste comparando com `math.log` e a diferença finita do E2.

E4 — Por que zerar o gradiente?

No `nn.py`, **comente** a linha `model.zero_grad()` e rode de novo. 1. O que acontece com a loss ao longo dos passos? 2. Explique usando o fato de que os gradientes são acumulados com `+=`. 3. Relacione com o `W.grad = None` do Capítulo 1.

E5 — Bate com o PyTorch? (desafio)

Escreva um script que monte a **mesma expressão** com `Value` e com `torch.tensor(..., requires_grad=True)`, rode `backward()` nos dois e compare `data` e todos os `grad`. 1. Eles batem (com tolerância de $1e-6$)? 2. Troque uma operação (ex.: use `relu` em vez de `tanh`) e confirme que ainda batem.

A solução de referência está em [solucoes/e5_check_vs_torch.py](#).

E6 — Mexa na arquitetura e na learning rate

No `nn.py`: 1. Troque `MLP(3, [4, 4, 1])` por `MLP(3, [8, 1])` e depois `MLP(3, [16, 16, 1])`. Quantos parâmetros cada um tem? A loss final muda? 2. Varie a learning rate (`0.05`) para `0.5` e `0.001`. O que

acontece com `0.5` (instável?) e com `0.001` (lento?)? 3. Troque a ativação `tanh` por `relu` nos neurônios. Treina melhor ou pior aqui?

E7 — Visualize o grafo (desafio, opcional)

Escreva uma função que percorra `_prev` recursivamente a partir de um `Value` e imprima o grafo (cada nó: seu `_op`, `data` e `grad`). Use-a numa expressão pequena para "ver" a estrutura que a `backward()` percorre. (Se conhecer a lib `graphviz`, desenhe de verdade — é opcional.)